

ANSWERS WITH RECEIPTS.

There are hundreds of "second brains" — note vaults with an AI bolted on, vector databases wearing a memory costume. Cortex is a different bet: a memory that has to **prove** every answer, out loud, or admit it doesn't know.

97%

RIGHT ON THE 185-LABEL RETRIEVAL EVAL

509 of 4,356

BLOCKS CROSSED OUT — AND KEPT ON THE PAGE

~3%

OF THE BRAIN A SESSION PAYS TO BOOT, BUDGET-ENFORCED IN CI

live figures from the running system, September 2026 • 128 notes
• plain markdown in a git repo

01 THE FIELD

What everyone else builds

The standard recipe: split your notes into chunks, turn each chunk into an embedding vector, store the vectors in a database, and when a question arrives,

paste the top-k "similar" chunks into the model's context and hope.

It demos beautifully. Then it fails in ways you can't see: the model answers confidently from chunks that didn't contain the answer, nothing tells you *which* note the claim came from, the index quietly drifts from the notes, and the context bill grows with every question.

- ✗ A confident wrong answer looks identical to a right one
- ✗ No source, no quote — provenance is a vibe
- ✗ The index is a lossy copy that rots away from the notes
- ✗ Corrections overwrite history, so stale answers are unfalsifiable
- ✗ Your data lives in someone's vector store, not in files you own

02

THE PIPELINE

One ask, end to end

Every `brain_ask` runs the same five steps. The only model in the loop reads **real note text** — never a summary, never an index — and the final check is deterministic code, not another model grading itself.



The shortlist exists to keep the read cheap — it never picks the answer. If the corpus doesn't contain one, the reply is NOT IN BRAIN, not a guess.

03

BM25, PLAINLY

The forty-year-old ranking trick that still wins

BM25 is how search engines ranked documents before neural anything: score each note against the question's words using three ideas that sound obvious once said. No training, no vectors, no drift — the notes themselves are the index.

Why not embeddings? Because we measured. The summary-index retriever this replaced scored **55%** on the eval; reading the actual text scores **97%**. We even tried blending BM25 with the fancier scoring — the blend *lost* (27.9% vs 29.1% recall@10 in the narrowing eval), so it stayed off. What ships is what measures better, not what demos better.

■ RARE WORDS WEIGH MORE

"restic" appears in three notes; "the" appears in all 128. A match on the rare word is worth far more — inverse document frequency.

■ REPETITION SATURATES

A note saying "backup" ten times isn't five times more relevant than one saying it twice. The score curves flat — term-frequency saturation.

■ LONG NOTES DON'T WIN BY VOLUME

A 20KB note naturally contains more matches. Scores normalize by length so a focused half-page can outrank an essay.

Every answer gets stamped

The answer arrives with its source note, a verbatim quote, and the git commit it was proven at. Then code — not a model — re-checks that the quote exists, byte for byte, in that file at that commit. The stamp is the verdict:

VERIFIED

CORRECTED

SUPERSEDED

UNVERIFIED

NOT IN BRAIN

SUPERSEDED means the quote is real but retracted — history, not current truth. A confident fabrication comes back UNVERIFIED instead of slipping through.

Corrections stay on the page

When a fact goes stale, it gets crossed out and superseded in place — never deleted. 509 of the brain's 4,356 blocks are retractions, kept precisely so a stale answer can be *caught quoting one*. Most systems overwrite; overwriting makes yesterday's wrong answer unfalsifiable.

55% → 97%

WHAT HAPPENED TO ACCURACY WHEN WE RETIRED THE SUMMARY INDEX AND MADE THE READER READ REAL TEXT

Budgets with teeth

Every session on every surface boots with the same small payload: who the operator is, a one-line router entry for all 128 notes, the live working state, and recent days — **9,335 of 358,858 tokens, about 3% of the brain**. Heavy write-days digest to a tag line instead of riding every boot.

Those numbers aren't a goal, they're **tests**. CI holds the boot under a fixed fraction of what a raw dump would cost and the router under an always-loadable ceiling — when tonight's writing pushed a budget over, the build went red and forced a real fix before anything shipped.

The lifetime math

Without a brain, every new session gets brought up to speed by hand — re-dropped PDFs, pasted code, re-explained context. With one, memory is read *server-side* and your session receives only the answer. The savings compound for the lifetime of the log, and the console reports them as an estimate that says so — no invented benchmarks, ever.

06

WHY OURS

Side by side

	THE USUAL SECOND BRAIN	CORTEX
PROOF	■ an answer, take it or leave it	■ note + verbatim quote + commit, re-checked by code, stamped
WHAT THE MODEL READS	■ embedded chunk soup	■ the actual note text — BM25 only shortlists
BEING WRONG	■ invisible	■ UNVERIFIED / NOT IN BRAIN, said out loud
CORRECTIONS	■ overwritten	■ crossed out and kept — stale answers get caught
STORAGE	■ a vendor's vector database	■ markdown files in a git repo you own

COST OVER TIME

■ grows with every question

■ boot and router budgets enforced in CI

UPKEEP

■ you, eventually, never

■ a nightly cloud groundskeeper fact-checks stale pages

OTHER AIS

■ no door

■ a budgeted, ask-only guest door with its own receipts

every claim on this page is either the mechanism itself, the measured 185-label eval, or live windowed telemetry – the only three things Cortex allows itself to cite.



CORTEX BY OBELYTH

[plain markdown](#) · [in git](#) · [with proof](#)